

**LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK**

“KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Siti Fadhilah Rahmi



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

1. Langkah Kerja Praktikum

- a. Langkah pertama yaitu, buka aplikasi yang dibutuhkan untuk praktikum (Eclipse) Lalu buat package pekan 3 dan berisi class Rekening, Main, Transaksi yang menggunakan kode program pada pekan sebelumnya.
- b. Langkah kedua, membuat kode program yang terdapat penambahan atau perubahan di dalam modul praktikum :

1. Class Rekening :

```
1. package Pekan_3;
2.
3. import java.util.ArrayList;
4.
5. public class Rekening {
6.     // 1, mengunci atribut dengan 'Private'
7.     private String nomorRekening;
8.     private String namaPemilik;
9.     private double saldo;
10.    private String pin; // data sensitif
11.
12.    // Implementasi Asosiasi (1-to-many)
13.    ArrayList<Transaksi>riwayatTransaksi;
14.
15.    // 2. Modifikasi Constructor untuk menerima PIN awal
16.    public Rekening(String nomor, String nama, double saldoAwal,
17.        String pinAwal) {
18.        this.nomorRekening = nomor;
19.        this.namaPemilik = nama;
20.        this.setSaldo(saldoAwal);
21.
22.        // Validasi PIN di dalam constructor
23.        if (pinAwal.length() == 6) {
24.            this.pin = pinAwal;
25.        } else {
26.            System.out.println("Peringatan : PIN harus 6
27.            digit! Menggunakan PIN default 123456");
28.            this.pin = "123456";
29.        }
30.    }
31.
32.    // 3. Getter untuk atribut yang diizinkan dibaca publik
33.    public String getNomorRekening() {
34.        return nomorRekening;
35.    }
36.}
```

```

34.
35.         public String getNamaPemilik() {
36.             return namaPemilik;
37.         }
38.
39.         // 4. Method Otentikasi Internal (Validasi
    Enkapsulasi()
40.         public boolean otentikasi(String inputPin) {
41.             return this.pin.equals(inputPin);
42.         }

```

Kode ini merupakan class **Rekening** dalam package **Pekan_3**, yang merupakan versi pengembangan dari kode sebelumnya dengan menerapkan konsep **enkapsulasi**. Perbedaan utamanya terletak pada atribut nomorRekening, namaPemilik, saldo, dan pin yang kini dideklarasikan sebagai **private**, sehingga tidak dapat diakses langsung dari luar class dan hanya bisa dimanipulasi melalui method tertentu, guna melindungi keamanan dan integritas data, terutama data sensitif seperti pin. Atribut **riwayatTransaksi** tetap dipertahankan sebagai implementasi asosiasi satu-ke-banyak antara rekening dan transaksi.

Pada **constructor Rekening**, parameter yang diterima kini bertambah dengan **pinAwal**, sehingga saat objek rekening dibuat, PIN awal juga harus disertakan. Nilai saldoAwal tidak langsung diberikan ke atribut saldo, melainkan melalui pemanggilan method `setSaldo(saldoAwal)`, yang mengindikasikan adanya validasi tambahan pada proses pengisian saldo (meski implementasi method tersebut tidak ditampilkan). Selain itu, constructor juga melakukan validasi terhadap panjang PIN yang dimasukkan; jika PIN berjumlah 6 digit maka akan disimpan sebagai pin rekening, namun jika tidak sesuai, sistem akan menampilkan peringatan dan secara otomatis menggunakan PIN default "123456" sebagai bentuk penanganan kesalahan input.

Selanjutnya terdapat dua method **getter**, yaitu `getNomorRekening()` dan `getNamaPemilik()`, yang berfungsi untuk mengizinkan pembacaan nilai atribut privat nomorRekening dan namaPemilik dari luar class tanpa harus membuka

akses langsung ke atributnya, sesuai prinsip enkapsulasi bahwa hanya atribut tertentu yang boleh diakses publik.

Terakhir, terdapat method **otentikasi(String inputPin)** yang berfungsi sebagai validasi keamanan internal, dengan cara membandingkan nilai inputPin yang dimasukkan pengguna dengan nilai pin yang tersimpan di dalam objek menggunakan `.equals()`. Method ini mengembalikan nilai boolean true jika PIN cocok dan false jika tidak, sehingga dapat digunakan sebagai syarat sebelum mengizinkan proses transaksi seperti penarikan tunai, tanpa pernah mengekspos nilai pin secara langsung ke luar class.

Mengapa kita tidak membuat method `setSaldo()` dan `getPin()` ?

Karna, Saldo adalah atribut yang nilainya harus berubah melalui **aturan bisnis tertentu**, bukan diubah sembarangan. Jika kita membuat method `setSaldo()` yang bisa dipanggil bebas dari luar class, maka siapa pun bisa mengubah saldo. Itu sebabnya perubahan saldo hanya boleh terjadi melalui method seperti `setorTunai()` dan `tarikTunai()`, yang di dalamnya sudah ada validasi (nominal harus positif, saldo mencukupi, dan sebagainya). Kalaupun ada method `setSaldo()` seperti yang dipanggil di constructor, sifatnya biasanya `private`. Sedangkan pin, Karena **pin adalah data sensitif**. Jika dibuatkan getter publik, maka nilai pin bisa "dibocorkan" dan dibaca oleh class atau pihak lain di luar Rekening, yang jelas berbahaya untuk keamanan data nasabah. Prinsip enkapsulasi di sini bukan cuma "menyembunyikan atribut", tapi juga **membatasi cara mengaksesnya**. Karena itu, alih-alih menyediakan `getPin()` untuk membaca pin secara langsung, dibuatlah method `otentikasi(String inputPin)` yang hanya mengembalikan hasil **perbandingan** (true/false), tanpa pernah mengekspos nilai pin yang sebenarnya. Dengan begitu, sistem tetap bisa memverifikasi PIN pengguna tanpa risiko nilai pin bisa "dicuri" atau dibaca langsung oleh kode dari luar.

2. Class Main

Untuk kelas Main tidak ada perubahan dari modul, tetapi terdapat perubahan pada bagian tugas/latihan praktikum. Jadi kelas main masih sama seperti pada pekan sebelumnya

3. Class Transaksi

```
1. package Pekan_3;
2.
3. public class Transaksi {
4.     // 1. Mengubah semua atribut menjadi private
5.     private String idTransaksi;
6.     private String jenis;
7.     private double nominal;
8.
9.     // Costructor
10.    public Transaksi(String id, String jenis, double nominal) {
11.        this.idTransaksi = id;
12.        this.jenis = jenis;
13.        this.nominal = nominal;
14.    }
15.
16.    // 2. Hanya menyediakan getter (Read-only)
17.    public String getIdTransaksi() {
18.        return idTransaksi;
19.    }
20.
21.    public String getJenis() {
22.        return jenis;
23.    }
24.
25.    public double getNominal() {
26.        return nominal;
27.    }
28.
29.    public void cetakDetail() {
30.        System.out.println("ID : " + idTransaksi + " | Jenis : "
31.            + jenis + " | Nominal : Rp" + nominal);
32.    }
```

Kode ini merupakan class Transaksi dalam package Pekan_3, yang merupakan versi enkapsulasi dari class Transaksi sebelumnya. Ketiga atribut, yaitu idTransaksi, jenis, dan nominal, kini dijadikan private, sehingga tidak dapat diakses atau diubah langsung dari luar class, dengan tujuan menjaga agar data transaksi tetap konsisten dan tidak bisa dimanipulasi sembarangan setelah objek dibuat. Constructor tetap berfungsi sama seperti sebelumnya, yaitu

menerima parameter id, jenis, dan nominal untuk menginisialisasi ketiga atribut tersebut saat objek Transaksi dibuat.

Untuk mengakses nilai-nilai atribut privat tersebut, disediakan tiga method getter, yaitu getIdTransaksi(), getJenis(), dan getNominal(), yang sifatnya read-only atau hanya untuk membaca nilai, tanpa disertai method setter. Hal ini sengaja dilakukan karena data transaksi seharusnya bersifat immutable (tidak dapat diubah) setelah tercatat, sebab representasi riwayat transaksi seharusnya tetap dan tidak boleh diedit demi menjaga keakuratan catatan keuangan. Terakhir, method cetakDetail() tetap dipertahankan untuk menampilkan informasi transaksi secara lengkap, meliputi ID, jenis, dan nominal transaksi.

2. Tugas/Latihan Praktikum

- a. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).
- b. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
 - Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
 - Jika mengembalikan nilai `true`, jalankan transaksi.
 - Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Jawaban :

- a. Kode program class Main :

```
1. case 1 :
2.     System.out.print("Masukkan No Rekening: ");
3.     String no = input.nextLine();
4.     System.out.print("Masukkan Nama Pemilik : ");
5.     String nama = input.nextLine();
6.     System.out.print("Masukkan Saldo Awal : ");
7.     double saldo = input.nextDouble();
8.     input.nextLine();
9.     System.out.println("Masukkan PIN Awal : ");
10.    String pin = input.nextLine();
11.    // Instansiasi Objek / Menjalankan Constructor
12.    akunAktif = new Rekening(no, nama, saldo, pin);
13.
14.    rekeningAktif.add(akunAktif);
15.    System.out.println("Rekening berhasil ditambahkan dan
    ditumpuk. Total rekening: " + rekeningAktif.size());
16.
17.    Break;
```

Kode ini merupakan potongan **case 1** dari program `Pekan_3`, yang berfungsi untuk menambahkan rekening baru. Program meminta input dari pengguna berupa nomor rekening, nama pemilik, dan saldo awal menggunakan `input.nextLine()` dan `input.nextDouble()`. Setelah input saldo (yang bertipe `double`) dibaca, ditambahkan `input.nextLine()` tambahan yang berfungsi untuk

membersihkan sisa karakter newline di buffer input, agar input String berikutnya (pin) tidak terlewat atau terbaca kosong akibat sisa enter dari `nextDouble()`.

Perbedaan utama dari versi `Pekan_2` adalah adanya input tambahan berupa **pin**, yang diminta melalui `input.nextLine()`, sesuai dengan constructor `Rekening` pada `Pekan_3` yang kini mewajibkan parameter pin saat objek dibuat. Selanjutnya, objek `akunAktif` diinstansiasi dengan memanggil constructor `new Rekening(no, nama, saldo, pin)`, lalu dimasukkan ke dalam list `rekeningAktif` menggunakan `.add()`. Terakhir, program menampilkan pesan konfirmasi bahwa rekening berhasil ditambahkan beserta jumlah total rekening yang tersimpan, sebelum keluar dari case menggunakan `break`.

b. Kode program class Main :

```
1. case 3 :
2.     System.out.println("Masukkan PIN : ");
3.     String pinYangDiinput = input.nextLine();
4.     if (akunAktif.otentikasi(pinYangDiinput) == false) {
5.         System.out.println("Akses Ditolak: PIN yang Anda
6.         masukkan salah!");
7.     } else {
8.         if (akunAktif.getSaldo() < 10000) {
9.             System.out.println("Transaksi Gagal: Saldo tidak
10.             mencukupi. Saldo Anda: Rp" + akunAktif.getSaldo());
11.         } else {
12.             System.out.print("Masukkan nominal tarik : ");
13.             double tarik = input.nextDouble();
14.             akunAktif.tarikTunai(tarik);
15.         }
16.     }
17.     break;
18. case 6 :
19.     System.out.print("Masukkan PIN : ");
20.     String pinYangDiinput1 = input.nextLine();
21.     if (akunAktif.otentikasi(pinYangDiinput1) == false) {
22.         System.out.println("Akses Ditolak: PIN yang Anda
23.         masukkan salah!");
24.     } else {
25.         akunAktif.cetakMutasi();
26.     }
27.     break;
```

Kode ini merupakan potongan **case 3** (tarik tunai) dan **case 6** (cetak mutasi) dari program `Pekan_3`, yang kini menambahkan lapisan **keamanan berupa verifikasi PIN** sebelum transaksi sensitif dapat dilakukan. Pada kedua case, program terlebih dahulu meminta pengguna memasukkan PIN melalui `input.nextLine()`, kemudian nilai tersebut dicek menggunakan method `otentikasi()` milik objek `akunAktif`. Jika hasil otentikasi bernilai `false` (PIN salah), maka sistem akan menampilkan pesan "Akses Ditolak" dan proses transaksi tidak dilanjutkan.

Pada **case 3**, jika PIN terbukti benar, program akan melanjutkan ke pengecekan saldo menggunakan `getSaldo()` (bukan mengakses atribut saldo secara langsung, karena kini bersifat `private`). Jika saldo kurang dari 10.000, transaksi dinyatakan gagal, namun jika mencukupi, pengguna diminta memasukkan nominal penarikan yang kemudian diproses melalui method `tarikTunai()`. Pada **case 6**, jika PIN valid, program akan langsung memanggil method `cetakMutasi()` untuk menampilkan riwayat transaksi rekening tersebut.

Perbedaan utama dari versi `Pekan_2` terletak pada penerapan konsep **enkapsulasi dan keamanan data**, di mana akses ke atribut saldo kini melalui getter (`getSaldo()`), serta transaksi penting seperti tarik tunai dan pengecekan mutasi tidak lagi bisa dilakukan secara bebas, melainkan harus melalui proses verifikasi PIN terlebih dahulu.

OUTPUT :

a. Output soal pertama

Case 1 :

```
=== SISTEN PERBANKAN MINI ===  
  
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi  
Pilih Menu : 1  
Masukkan No Rekening: 000  
Masukkan Nama Pemilik : Fedo  
Masukkan Saldo Awal : 200000  
Masukkan PIN : 060407  
Rekening berhasil ditambahkan dan ditumpuk. Total rekening: 1
```

b. Output soal kedua

Case 3 :

```
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi  
Pilih Menu : 3  
Masukkan PIN : 060407  
Masukkan nominal tarik : 50000  
Tarik tunai Rp 50000.0 berhasil. Saldo saat ini : Rp150000.0
```

Case 6 :

```
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi  
Pilih Menu : 6  
Masukkan PIN : 060407  
ID : TRX-T-1790408186320 | Jenis : Debit | Nominal : Rp50000.0
```